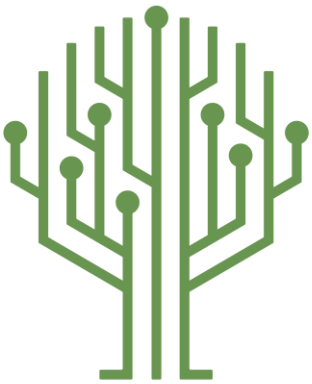


Green Pace

Green Pace Secure Development Policy

Contents

Overview 2
Purpose 2
Scope 2
Module Three Milestone 2
- Ten Core Security Principles 2
- C/C++ Ten Coding Standards 3
  - Coding Standard 1 4
  - Coding Standard 2 5
  - Coding Standard 3 7
  - Coding Standard 4 8
  - Coding Standard 5 9
  - Coding Standard 6 10
  - Coding Standard 7 12
  - Coding Standard 8 14
  - Coding Standard 9 16
  - Coding Standard 10 17
- Defense-in-Depth Illustration 19

Project One 19
1. Revise the C/C++ Standards 19
2. Risk Assessment 19
3. Automated Detection 19
4. Automation 19
5. Summary of Risk Assessments 20
6. Create Policies for Encryption and Triple A 21
7. Map the Principles 21

Audit Controls and Management 23
Enforcement 23
Exceptions Process 23
Distribution 24
Policy Change Control 24
Policy Version History 24
Appendix A Lookups 24
- Approved C/C++ Language Acronyms 24

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and auditing standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): [Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines](#).

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	Input validation is to ensure a user's input is clean and correct before processing the data any further to prevent malicious attacks.
2. Heed Compiler Warnings	Compiler warnings usually flag potential bugs or security risk that a developer might have missed. For an example it will warn you when your scanner is left open which could consume resources or lead to vulnerability.
3. Architect and Design for Security Policies	Architect and Design for Security Policies is a methodology of breaking down multiple parts of a program to design a web that hopefully will keep out intruders in case something gets breached. A common example of this could be Sever side vs client side. If the client side is breached it doesn't put ALL users at risk as the server holds most of the data and buys developers precious time to fix that vulnerability and kick out any intruders.
4. Keep It Simple	Keeping it simple is a design principle that suggests simplicity should be a key goal at all stages of development as it allows for a better development experience for everyone involved. It aims to prevent over-engineering and unnecessary complexity.
5. Default Deny	Default deny tells you to block ALL access to a network or system unless you specifically allow it. This is used to minimize potential vulnerabilities in an attack.
6. Adhere to the Principle of Least Privilege	Adhere to the Principle of Least Privilege tells you to always do all processes using the minimal access rights needed. This could be used for safety mainly. You could think of a real-world example of this would be giving a manager of a store access to the store codes in their assigned store but not the access code to the entire company and every single store.
7. Sanitize Data Sent to Other Systems	Sanitize Data Sent to Other Systems means to clean and validate all data before sending it to another program or a database as it could cause errors or leave

Principles	Write a short paragraph explaining each of the 10 principles of security.
	vulnerabilities.
8. Practice Defense in Depth	Practicing defense in depth means using multiple layers of security and ensuring to test it. For instance, a user could only edit their settings, however an admin can change everyone's settings. So, it's important to ensure that users' data changes when they want it to and admins can change their data as well as users' data.
9. Use Effective Quality Assurance Techniques	Using Effective Quality Assurance Techniques means to do effective QA for the program in development. This could be by creating J unit test or preprogrammed inputs to test a certain program.
10. Adopt a Secure Coding Standard	This is a rule that just is a warning to adopt ANY coding good coding standard. You need to have at least ONE which is commonly OWASP.

C/C++ Ten Coding Standards

Complete the coding standards portion of the template according to the Module Three milestone requirements. In Project One, follow the instructions to add a layer of security to the existing coding standards. Please start each standard on a new page, as they may take up more than one page. The first seven coding standards are labeled by category. The last three are blank so you may choose three additional standards. Be sure to label them by category and give them a sequential number for that category. Add compliant and non-compliant sections as needed to each coding standard.

Coding Standard 1

Coding Standard	Label	Name of Standard
Data Type	[STD-001-CPP]	Use correct Data types that prevent overflow, truncation, or unexpected program behavior.

Noncompliant Code

Int is the wrong data type so it will cause errors.
Int Hello = Hello;

Compliant Code

String is the correct data type for input hello
String Hello = Hello;

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Using correct data types ensures safe handling of data and prevents overflow errors.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Low	Medium	3

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Memory check	Detects overflow and unsafe types
SonarQube	10.x	Security rules	Detects bad coding practices
Clang Static Analyzer	Latest	Static analysis	Finds runtime bugs
Visual Studio Analyzer	Latest	Code warnings	Detects type issues

Coding Standard 2

Coding Standard	Label	Name of Standard
Data Value	[STD-002-CPP]	Programs should check the data value that falls inside an acceptable range.

Noncompliant Code

This code doesn't check if the data entered by the user is valid.

```
Int age;
Std::cin >> age;
```

Compliant Code

This code checks if the value is entered in an acceptable range

```
int age;
std::cin >> age;

if (age >= 0 && age <= 120) {
   std::cout << "Valid age";
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Checking values ensures only valid data is used and prevents errors or attacks.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	High	Low	High	4

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Input Validation	Detects Invalid Value Usage
SonarQube	10.x	Security rules	Detects missing Validation
Clang Static Analyzer	Latest	Static analysis	Finds runtime bugs
Visual Studio Analyzer	Latest	Code warnings	Detects type issues

Coding Standard 3

Coding Standard	Label	Name of Standard
String Correctness	[STD-003-CPP]	Unsafe strings functions can lead to bugger overflow or memory corruption. Use safe string handling methods.

Noncompliant Code

Char only has limited size and doesn't check the size buffer.
char name[10]; gets(name);

Compliant Code

This code is a safe method that prevents more overflow as string auto allocates space.
std::string name; std::getline(std::cin, name);

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Using safe strings prevents buffer overflow and keeps code safe.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	Medium	High	5

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Buffer Overflow	Detects unsafe Vulnerabilities
SonarQube	10.x	Security hotspots	Detects String Vulnerabilities
Clang Static Analyzer	Latest	Static analysis	Finds Memory Issues
Visual Studio Analyzer	Latest	Code warnings	Detects Unsafe Function

Coding Standard 4

Coding Standard	Label	Name of Standard
SQL Injection	[STD-004-CPP]	User input should NEVER be directly included in SQL queries. It needs proper validation and prepared statements to prevent SQL injection.

Noncompliant Code

This code directly inserts users input into SQL query
std::string query = "SELECT * FROM users WHERE name = '" + username + "'";

Compliant Code

This code uses parameters and has a safe statement to pass user input.
PreparedStatement stmt = connection.prepareStatement("SELECT * FROM users WHERE name = ?"); stmt.setString(1, username);

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Prepared statements prevent attackers from injecting SQL commands.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	Medium	High	5

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	SQL Injection Rules	Detects unsafe Queries
SonarQube	10.x	Input Check	Detects unsafe input usage
Clang Static Analyzer	Latest	Static analysis	Finds Vulnerabilities
Visual Studio Analyzer	Latest	Code warnings	Detects bad SQL

Coding Standard 5

Coding Standard	Label	Name of Standard
Memory Protection	[STD-005-CPP]	Improper memory handling could allow attackers to overwrite memory and execute malicious code.

Noncompliant Code

This code copies data into a buffer without checking size
char buffer[10]; strcpy(buffer, input);

Compliant Code

This code checks the buffer size before copying data.
char buffer[10]; strncpy(buffer, input, sizeof(buffer) - 1);

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Safe memory handling prevents attackers from exploiting memory errors.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Medium	High	4

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Memory Leaks	Detects unsafe Memory Use
SonarQube	10.x	Security Rules	Detects Unsafe Memory Practices
Clang Static Analyzer	Latest	Static analysis	Finds Memory Bugs
Valgrind	Latest	Memory Checker	Detects memory leaks and errors

Coding Standard 6

Coding Standard	Label	Name of Standard
Assertions	[STD-006-CPP]	Assertions help developers detect errors during development by verifying that certain conditions are true. They improve program reliability

Noncompliant Code

This code assumes the value is valid without checking.

```
int divide(int a, int b) {
   return a / b;
}
```

Compliant Code

This code uses an assertion to ensure the value is valid before performing operation

```
#include <cassert>

int divide(int a, int b) {
   assert(b != 0);
   return a / b;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Assertions help catch errors early during development.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Low	Medium	Low	Low	2

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Logical Check	Detects Missing Assertion
SonarQube	10.x	Code Quality	Detects Weak Validation
Clang Static Analyzer	Latest	Static analysis	Findings Logical Issues

Tool	Version	Checker	Description Tool
Visual Studio Analyzer	Latest	Code warnings	Detects missing Checks

Coding Standard 7

Coding Standard	Label	Name of Standard
Exceptions	[STD-007-CPP]	Exceptions allow programs to handle errors in a structured way. Proper handling prevents crashing

Noncompliant Code

This code does not handle potential runtime error

```
int result = a / b;
```

Compliant Code

This code uses exception handling to catch errors

```
try {
   int result = a / b;
}
catch (...) {
   std::cout << "Error occurred";
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Proper exception handling prevents crashes and improves stability.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Medium	Low	Medium	3

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Runtime Issues	Detects unsafe Operations
SonarQube	10.x	Exception Handling	Detects Poor Handling
Clang Static Analyzer	Latest	Static analysis	Finds crash Risks
Visual Studio Analyzer	Latest	Code Warnings	Detects Missing Handling

Coding Standard 8

Coding Standard	Label	Name of Standard
Input Validation	[STD-008-CPP]	User input should always be validated before being used by a program. Invalid or malicious input can cause errors.

Noncompliant Code

This code accepts user input without checking whether it is valid.

```
int number;
std::cin >> number;

std::cout << "You entered: " << number;
```

Compliant Code

This code checks that the input is valid before using it.

```
int number;

if (std::cin >> number) {
   std::cout << "You entered: " << number;
}
else {
   std::cout << "Invalid input.";
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Validating input prevents invalid or malicious data.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	Low	High	5

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Input Check	Detects Unsafe Input
SonarQube	10.x	Input Validation	Detects Missing Validation

Tool	Version	Checker	Description Tool
Clang Static Analyzer	Latest	Static analysis	Finding Logic Issues
Visual Studio Analyzer	Latest	Code Warnings	Detects Bad Input Handling

Coding Standard 9

Coding Standard	Label	Name of Standard
Pointer Safety Coding Standard	[STD-009-CPP]	Uninitialized pointers point to random memory location and cause undefined behavior. Initialize ALL pointers to ensure the memory is accessed safely

Noncompliant Code

This code declares a pointer but does not initialize it before use
int* ptr; *ptr = 10;

Compliant Code

This code initializes the pointer before it is used.
int value = 10; int* ptr = &value;

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Initializing pointers prevents crashes and unsafe memory access.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Medium	High	4

Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Pointer Check	Detects uninitialized Pointers
SonarQube	10.x	Security Rules	Detects Unsafe Pointer Usage
Clang Static Analyzer	Latest	Static analysis	Find pointer bugs
Valgrind	Latest	Memory Checker	Detects Invalid Memory Access

Coding Standard 10

Coding Standard	Label	Name of Standard
Integer Overflow Coding Standard	[STD-010-CPP]	Integer overflow occurs when a calculation produces a value larger than a data type can store. This causes undefined behavior and could overwrite memory.

Noncompliant Code

This code performs addition without checking if the result exceeds the allowed range.

```
int a = 2000000000;
int b = 2000000000;

int result = a + b;
```

Compliant Code

This code checks for overflow before performing the calculation.

```
int a = 2000000000;
int b = 2000000000;

if (a > INT_MAX - b) {
   std::cout << "Overflow detected";
}
else {
   int result = a + b;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s): Checking limits prevents overflow and incorrect calculations.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Medium	Medium	High	4

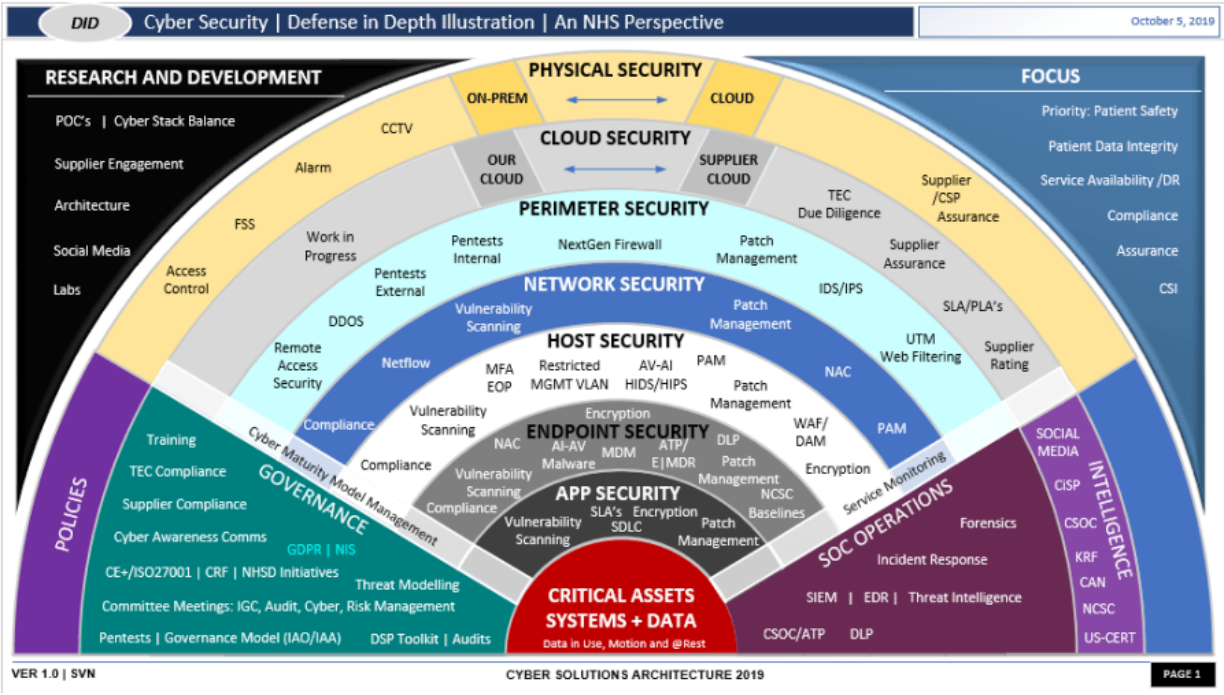
Automation

Tool	Version	Checker	Description Tool
CppCheck	2.10	Overflow Check	Detects integer Overflow

Tool	Version	Checker	Description Tool
SonarQube	10.x	Security Rules	Detects Unsafe Math
Clang Static Analyzer	Latest	Static analysis	Finds Overflow Risks
Visual Studio Analyzer	Latest	Coding Warnings	Detects Arithmetic Issues

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

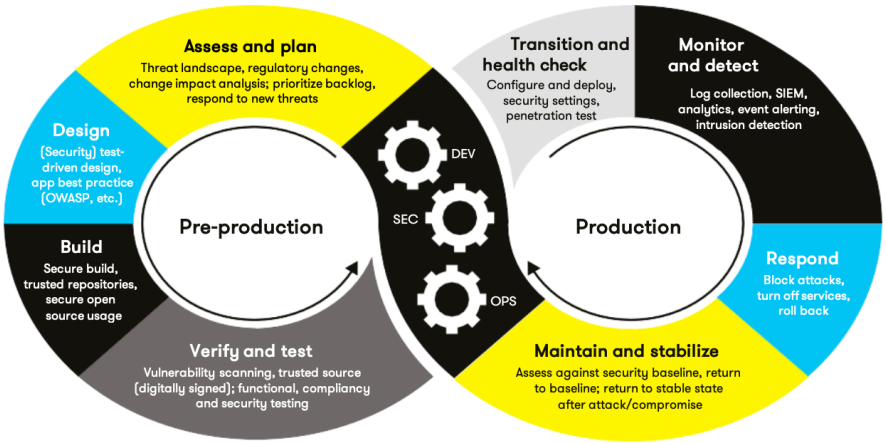
Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description—if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well-established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

Automation will be used throughout the DevSecOps pipeline to enforce secure coding standards. During the Create phase, developers will use IDE tools such as Visual Studio Analyzer to detect issues while writing code. In the Verify phase, tools like Cppcheck, SonarQube, and Clang Static Analyzer will automatically scan for vulnerabilities such as memory issues, SQL injection, and unsafe input handling. During pre-production, testing methods like fuzzing and integration testing will identify runtime issues. Before Release, security checks such as code signing and integrity checks will ensure only secure code is deployed. This process supports defense in depth by finding and fixing problems early on in development.

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Unlikely	Medium	High	2
STD-002-CPP	Medium	High	Low	High	4
STD-003-CPP	High	High	Medium	High	5
STD-004-CPP	High	High	Medium	High	5
STD-005-CPP	High	Medium	Medium	High	4
STD-006-CPP	Low	Medium	Low	Low	2
STD-007-CPP	Medium	Medium	Low	Medium	3
STD-008-CPP	High	High	Low	High	5
STD-009-CPP	High	Medium	Medium	High	4
STD-010-CPP	High	Medium	Medium	High	4
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided**.**

a. Explain each type of encryption, how it is used, and why and when the policy applies.
b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	**Explain what it is and how and why the policy applies.**
Encryption at rest	Encryption at rest protects stored data such as files and databases. It ensures that even if data is accessed without permission, it cannot be read. This is important for protecting sensitive user information.
Encryption in flight	Encryption in flight protects data while it is being sent over a network. This is typically done using HTTPS or TLS. It prevents attackers from intercepting and reading data during transmission.
Encryption in use	Encryption in use protects data while it is being processed in memory. This helps prevent attackers from accessing sensitive data during execution.

b. Triple-A Framework*	**Explain what it is and how and why the policy applies.**
Authentication	Authentication verifies the identity of users through login systems such as usernames and passwords. It ensures only authorized users can access the system.
Authorization	Authorization determines what a user is allowed to do after logging in. It ensures users only access data and features based on their role.
Accounting	Accounting tracks user activity such as logins, file access, and changes to the system. This helps detect suspicious behavior and supports security monitoring.

***Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may

have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.

Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	08/05/2020	Initial Template	David Buksbaum	
1.1	4/7/2025	Added Coding standards & Project 1	Justin	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV